

Meine Favoriten – Technische Dokumentation

Version 0.7 vom 06.05.2026 / © MD. 2026

1. Übersicht

„Meine Favoriten“ ist eine clientseitige Single-Page-Application (SPA) ohne Backend. Sie besteht aus drei Dateien und benötigt keinen Build-Prozess, kein Framework und keine externe Abhängigkeit.

1.1 Dateistruktur

```
Favoriten-PWA/  
├─ index.html           # Struktur und Dialoge  
├─ app.js               # Gesamte Anwendungslogik  
├─ styles.css           # Styling  
├─ manifest.json        # PWA-Manifest  
├─ sw.js                # Service Worker  
├─ BEDIENUNGSANLEITUNG.pdf  
├─ icons/  
│   └─ icon-192.png  
│   └─ icon-512.png
```

1.2 Technologie-Stack

Schicht	Technologie
Markup	HTML5 (<code><dialog></code> , <code><datalist></code> , semantische Elemente)
Logik	Vanilla JavaScript (ES2022, keine Frameworks)
Styling	CSS3 (Custom Properties, Flexbox, Grid)
Persistenz	<code>localStorage</code> (Browser-API)
Offline	Service Worker (Cache-first-Strategie)
Installation	PWA Manifest (<code>manifest.json</code>)

2. Datenmodell

Alle Daten werden im `localStorage` des Browsers unter drei Schlüsseln gespeichert.

2.1 localStorage-Schlüssel

Konstante	Schlüssel	Typ	Inhalt
<code>STORAGE_KEY</code>	<code>favorite-links-v1</code>	<code>Array<Link></code>	Alle Favoriten
<code>CATEGORY_ORDER_KEY</code>	<code>favorite-category-order-v1</code>	<code>Array<string></code>	Gewünschte Kategorienreihenfolge
<code>VISIT_COUNTS_KEY</code>	<code>favorite-visit-counts-v1</code>	<code>Record<id, number></code>	Besuchszähler je Link-ID
–	<code>autoExportEnabled</code>	<code>"true" "false"</code>	Auto-Export-Einstellung

2.2 Link-Objekt

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "Tagesschau",
  "url": "https://www.tagesschau.de",
  "category": "Nachrichten"
}
```

Feld	Typ	Pflicht	Beschreibung
<code>id</code>	<code>string</code> (UUID v4)	ja	Eindeutige ID, generiert via <code>crypto.randomUUID()</code>
<code>title</code>	<code>string</code>	ja	Anzeigenname der Kachel
<code>url</code>	<code>string</code>	ja	Beliebiges URL-Schema (https, http, obsidian, ...)
<code>category</code>	<code>string</code>	ja	Kategoriezuordnung; Default: <code>"Allgemein"</code>

2.3 Export-Format (JSON)

```
{
  "exportedAt": "2026-05-06T14:30:22.000Z",
  "categoryOrder": ["Nachrichten", "Wissen", "Tools"],
  "links": [ { "id": "...", "title": "...", "url": "...", "category": "..." } ],
  "visitCounts": { "550e8400-...": 42 }
}
```

3. Architektur

3.1 Initialisierungsablauf

Seitenload

- |
- |— localStorage lesen → links, categoryOrder, visitCounts laden
- |— autoExportSuppressed = true (verhindert Export beim Start)
- |— saveLinks(links) (Normalisierung sicherstellen)
- |— update() (Render-Zyklus)
- |— autoExportSuppressed = false (Export ab jetzt erlaubt)
- |— hasChanges = false

3.2 Render-Zyklus

Jede Zustandsänderung läuft durch dieselbe Pipeline:

Nutzeraktion

- |
- |— Daten mutieren (links / categoryOrder / visitCounts)
- |— localStorage schreiben (safeLocalStorageSetItem)
- |— scheduleAutoExport() (Debounce-Timer neu starten)
- |— update()
 - |— syncCategoryOrder() (neue Kategorien einpflegen)
 - |— pruneVisitCounts() (verwaiste Zähler entfernen)
 - |— renderTopStats() (Statistik-Panel aktualisieren)
 - |— render() (Kachelbereich neu aufbauen)

3.3 Globale Zustandsvariablen

Variable	Typ	Beschreibung
<code>links</code>	<code>Array<Link></code>	Aktuell geladene Favoritenliste
<code>categoryOrder</code>	<code>Array<string></code>	Gewünschte Anzeigereihenfolge
<code>visitCounts</code>	<code>Record<id, number></code>	Besuchszähler
<code>searchQuery</code>	<code>string</code>	Aktueller Suchbegriff (lowercase)
<code>hasChanges</code>	<code>boolean</code>	Ungespeicherte Änderungen vorhanden
<code>statsVisible</code>	<code>boolean</code>	Statistik-Panel sichtbar
<code>draggedId</code>	<code>string null</code>	ID der aktuell gezogenen Kachel
<code>editingLinkId</code>	<code>string null</code>	ID des gerade bearbeiteten Links
<code>activeCategoryMenu</code>	<code>HTMLElement null</code>	Aktuell offenes Kategorien-Menü
<code>pendingCategoryOrder</code>	<code>Array<string></code>	Zwischenstand im Sortierdialog

4. Kernfunktionen

4.1 Speicher-Schicht

`safeLocalStorageSetItem(key, value)`

Wrapper um `localStorage.setItem` mit zweistufiger Fehlerbehandlung bei `QuotaExceededError`:

1. Verwaiste Einträge in `visitCounts` entfernen und erneut versuchen
2. Bei anhaltendem Fehler: kritische Warnung an den Nutzer

`loadLinks()` / `saveLinks()` / `setLinks()`

- `loadLinks()` liest und validiert aus dem Storage; gibt `defaultLinks` zurück falls leer
- `saveLinks()` serialisiert und schreibt, ruft danach `scheduleAutoExport()` auf
- `setLinks()` setzt `links` und ruft `saveLinks()` auf, setzt `hasChanges = true`

`loadCategoryOrder()` / `saveCategoryOrder()`

Lesen/Schreiben der Kategorienreihenfolge. `saveCategoryOrder()` setzt `hasChanges = true` und ruft `scheduleAutoExport()` auf.

`loadVisitCounts()` / `saveVisitCounts()`

`saveVisitCounts()` löst **keinen** Auto-Export aus (Besuche sollen den Export-Rhythmus nicht beeinflussen).

4.2 URL- und Kategorie-Normalisierung

`normalizeUrl(rawUrl)`

Ergänzt `https://` wenn kein Protokoll erkannt wird (Regex: `/^[a-zA-Z]+:/`). Unterstützt damit beliebige Schemata wie `obsidian://`, `file://`, `ftp://`.

`normalizeCategory(rawCategory)`

Trimmt Whitespace, gibt "Allgemein" zurück wenn leer.

4.3 Kategorien-Synchronisation

`syncCategoryOrder()`

Gleicht `categoryOrder` mit den tatsächlich in `links` vorhandenen Kategorien ab:

- Nicht mehr existierende Kategorien werden entfernt
- Neue (noch nicht geordnete) Kategorien werden per `unshift` vorne eingefügt
- `MOST_VISITED_CATEGORY` wird immer herausgefiltert (automatisch verwaltet)

`getExistingCategories()`

Gibt ein dedupliziertes Array aller Kategorien aus `links` zurück (ohne die automatische Kategorie).

4.4 Besuchszähler

`recordVisit(linkId)`

Inkrementiert den Zähler, setzt `suppressAutoExportOnce = true` (damit der Besuch keinen Export auslöst), und führt nach einem `setTimeout(0)` einen `update()` durch.

`pruneVisitCounts()`

Entfernt Zähler für IDs, die nicht mehr in `links` existieren.

`getMostVisitedLinks()`

Sortiert Links nach Besuchshäufigkeit absteigend (bei Gleichstand alphabetisch), gibt maximal `MOST_VISITED_LIMIT` (6) zurück.

4.5 Import-Validierung

`sanitizeImportedLinks(rawLinks)`

Filtert und bereinigt importierte Link-Objekte:

- Pflichtfelder `title` und `url` müssen Strings sein
- URL wird normalisiert und per `new URL()` validiert
- Fehlende `id` wird durch neue UUID ersetzt
- Ungültige Einträge werden still verworfen

4.6 Drag & Drop

`moveLink(fromId, toId, targetCategory)`

Verschiebt eine Kachel innerhalb derselben Kategorie. Kategorie-Wechsel wird abgelehnt (dafür ist `reassignLinkCategory` zuständig). Berechnet `adjustedToIndex` um den Versatz durch das Entfernen des Elements zu kompensieren.

`moveLinkToCategoryEnd(fromId, targetCategory)`

Verschiebt eine Kachel ans Ende einer Kategorie (Drop auf leeren Bereich).

`reassignLinkCategory(linkId, nextCategory)`

Ändert die Kategorie eines Links und fügt diese ggf. in `categoryOrder` ein.

5. Auto-Export-Mechanismus

5.1 Zustandsvariablen

Variable	Zweck
<code>autoExportEnabled</code>	Vom Nutzer ein-/ausschaltbar; persistiert in <code>localStorage</code>
<code>autoExportSuppressed</code>	Während Initialisierung <code>true</code> , danach <code>false</code>
<code>suppressAutoExportOnce</code>	Einmalige Unterdrückung (z. B. bei Besuchszähler-Updates)
<code>exportDebounceTimer</code>	Referenz auf den aktiven <code>setTimeout</code> -Handle
<code>autoExportTriggeredSinceLastChange</code>	Verhindert doppelten Export beim <code>beforeunload</code>

5.2 Ablauf

```

Änderung → scheduleAutoExport()
├─ Abbruch wenn: !autoExportEnabled || autoExportSuppressed ||
suppressAutoExportOnce
├─ autoExportTriggeredSinceLastChange = false
├─ clearTimeout(exportDebounceTimer) (Timer zurücksetzen)
└─ exportDebounceTimer = setTimeout(exportCurrentState, 5000)
                                └─ Nach 5s: exportCurrentState()
                                    └─ Payload bauen (links +
categoryOrder + visitCounts)
                                    └─ Blob → Object-URL →
<a>.click()
                                └─
autoExportTriggeredSinceLastChange = true

```

5.3 Sonderfall: Bearbeiten-Dialog

`openEditDialog()` ruft `clearTimeout(exportDebounceTimer)` auf. Der Timer wird erst nach `editForm`-Submit via `scheduleAutoExport()` neu gestartet. Damit können beliebig viele Bearbeitungen durchgeführt werden, ohne dass der Timer zwischendurch abläuft.

5.4 Export beim Verlassen (`beforeunload`)

Wenn `hasChanges === true` und `autoExportTriggeredSinceLastChange === false`, wird beim Schließen der Seite ein sofortiger Export ausgelöst.

6. Rendering

6.1 `render()`

Baut den gesamten Kachelbereich neu auf:

1. `tilesContainer.innerHTML = ""` (kompletter Reset)
2. Links nach aktuellem `searchQuery` filtern
3. Gefilterte Links nach Kategorie gruppieren (`Map<string, Link[]>`)
4. Automatische Kategorie „Am häufigsten besucht“ zuerst rendern
5. Dann alle Kategorien in `categoryOrder` -Reihenfolge

6.2 `renderCategorySection(category, items, options)`

Erstellt einen `<section>`-Block mit Header und Kachelgitter. Bei `isAutomatic = true` werden Drag & Drop und der Löschen-Button am Kategoriekopf deaktiviert.

6.3 `createTile(link, options)`

Factory-Funktion für eine einzelne Kachel (`<article>`). Erzeugt:

- Favicon (mit Mixed-Content-Schutz und SVG-Fallback)
- Titelzeile (Favicon + Name)
- URL-Zeile
- Aktionsleiste (Besuchs-Badge, Drag-Handle, , , )
- Drag & Drop Event Listener (nur bei nicht-automatischen Kacheln)

Favicon-Ladelogik:

```
URL ist https:// oder http://?  
├─ Ja: urlObj.protocol === 'http:' && location.protocol === 'https:?'  
│   └─ Ja: throw → SVG-Fallback (Mixed Content vermeiden)  
│       └─ Nein: favicon.src = `${origin}/favicon.ico`  
│           onerror → SVG-Fallback  
└─ Nein (custom scheme): SVG-Fallback
```

6.4 `renderCategorySuggestions()`

Befüllt das `<datalist>`-Element mit alphabetisch sortierten Kategorievorschlägen (`.sort(localeCompare("de"))`). Wird nach jedem `render()` aufgerufen.

6.5 renderTopStats()

Rendert das Statistik-Panel als innerHTML-String mit Balkengrafik (CSS- `width` in Prozent).
Wird nur neu gerendert wenn `statsVisible === true`.

7. Dialoge

Die App verwendet das native HTML5- `<dialog>` -Element für alle modalen Fenster.

7.1 Bearbeiten-Dialog (`#editDialog`)

- Geöffnet via `editDialog.showModal()`
- Formulardaten werden beim Submit validiert (leerer Titel, ungültige URL)
- Nach Speichern: `scheduleAutoExport()` explizit aufgerufen

7.2 Kategorien-sortieren-Dialog (`#sortCategoriesDialog`)

- Arbeitet auf `pendingCategoryOrder` (Kopie von `categoryOrder`)
- Drag & Drop innerhalb der Liste über HTML5 Drag API
- Vier Positions-Buttons je Eintrag (`⬆` `⬇` `⬇` `⬆`)
- Submit übernimmt `pendingCategoryOrder` → `categoryOrder`
- Abbrechen verwirft `pendingCategoryOrder` ohne Auswirkung

7.3 Kategorie-Kontextmenü

Kein `<dialog>`, sondern ein dynamisch erzeugtes `<div>` das direkt an `document.body` angehängt wird. Positionierung via `getBoundingClientRect()` mit Viewport-Kollisionserkennung (klappt nach oben wenn zu wenig Platz nach unten). Wird bei Klick außerhalb via `document.addEventListener("click")` geschlossen.

8. PWA-Implementierung

8.1 manifest.json

```
{
  "name": "Meine Favoriten",
  "short_name": "Favoriten",
  "start_url": "/index.html",
  "display": "standalone",
  "theme_color": "#317EFB",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192" },
    { "src": "/icons/icon-512.png", "sizes": "512x512" }
  ]
}
```

8.2 Service Worker (`sw.js`)

Cache-first-Strategie:

```
Fetch-Event
├─ Nur GET-Requests, nur same-origin
├─ Cache-Treffer? → direkt zurückgeben
└─ Kein Treffer → Netzwerk, Antwort in Cache schreiben
   └─ Netzwerk offline → Fallback auf /index.html
```

Beim `install`-Event werden alle App-Dateien vorsorglich gecacht (`addAll`). Beim `activate`-Event werden veraltete Caches gelöscht.

9. Sicherheit & Robustheit

9.1 Mixed Content

In `createTile()` wird geprüft ob `location.protocol === 'https:'` und `urlObj.protocol === 'http:'`. In diesem Fall wird die Favicon-Anfrage nicht gestellt und direkt auf den SVG-Fallback gewechselt, um Browser-Warnungen zu vermeiden.

9.2 localStorage-Quota

`safeLocalStorageSetItem()` fängt `QuotaExceededError` und versucht in zwei Stufen zu bereinigen, bevor eine Warnung ausgegeben wird.

9.3 Import-Validierung

`sanitizeImportedLinks()` validiert jeden importierten Eintrag einzeln mit `new URL()`. Ungültige Einträge werden verworfen, valide IDs werden beibehalten, fehlende IDs via `crypto.randomUUID()` neu generiert.

9.4 URL-Schema-Unterstützung

`normalizeUrl()` erkennt jedes bekannte Protokoll via `/^[a-zA-Z]+:/` und ergänzt nur dann `https://`, wenn kein Schema vorhanden ist. Damit werden `obsidian://`, `file://`, `ssh://` etc. korrekt durchgereicht.

10. Bekannte Einschränkungen

Einschränkung	Ursache	Workaround
Kein Favicon bei <code>http://</code> - Links auf HTTPS-Seite	Browser-seitige Mixed-Content-Policy	Globus-Icon als Fallback
PWA nicht installierbar über <code>file://</code>	Service Worker benötigt sicheren Ursprung	Lokalen Webserver verwenden
<code>localStorage</code> max. ~5–10 MB	Browser-Limit	Regelmäßiger Export; Bereinigung verwaister Zähler
Kein Server-seitiges Backup	By Design (offline-first)	Export-Funktion nutzen
<code>beforeunload</code> -Export nicht in allen Browsern garantiert	Browser-API-Einschränkung	Auto-Export aktiviert